LLM Inference Framework Comparison: vLLM vs SGLang

Pragati Rao — UID: 121332776
Rishabh Ranka — UID: 121084965

2025-12-17

Introduction

Large Language Models (LLMs) are increasingly deployed as online inference services, where system performance is determined not only by model quality but also by the efficiency of the serving framework. Key challenges in LLM serving include minimizing latency, maximizing throughput, and managing infrastructure cost under varying workloads.

This project evaluates two popular LLM inference frameworks: **vLLM** and **SGLang**, by deploying and benchmarking them on open-source language models. Specifically, we compare inference performance on **GPT-2**, a lightweight autoregressive model, and **TinyLlama**, a more modern instruction-tuned model. The goal is to analyze how different serving frameworks behave under different prompt workloads and generation lengths, and to identify best practices for efficient deployment.

Inference on GPT-2 Model using vLLM and SGLang

GPT-2 is a 124M-parameter transformer-based language model trained for next-token prediction. While it is not instruction-tuned or chat-optimized, its small size makes it suitable for controlled benchmarking experiments.

GPU Memory Usage

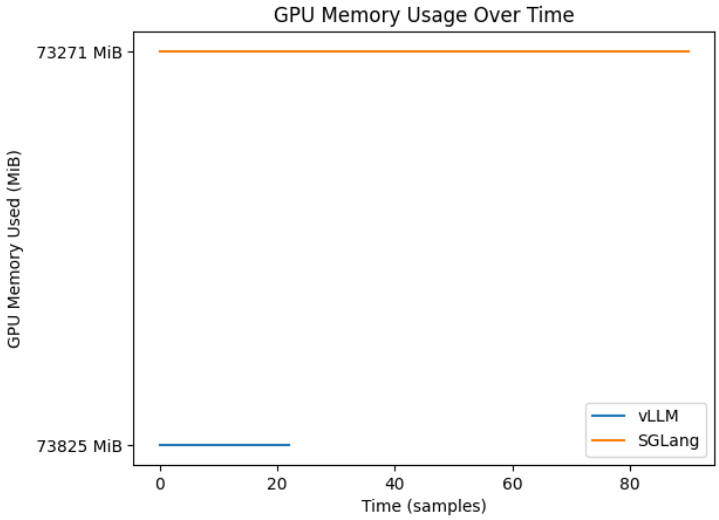

Figure 1: GPU Memory Usage for GPT-2

Both vLLM and SGLang exhibit stable GPU memory usage over time, indicating efficient memory management during inference. Memory is primarily allocated during model initialization rather than dynamically during request processing. This behavior is desirable in production systems, as it prevents memory fragmentation and reduces the risk of out-of-memory errors under sustained workloads. Memory efficiency is dominated by model size and KV-cache design rather than request volume for small models like GPT-2.

Inference Latency

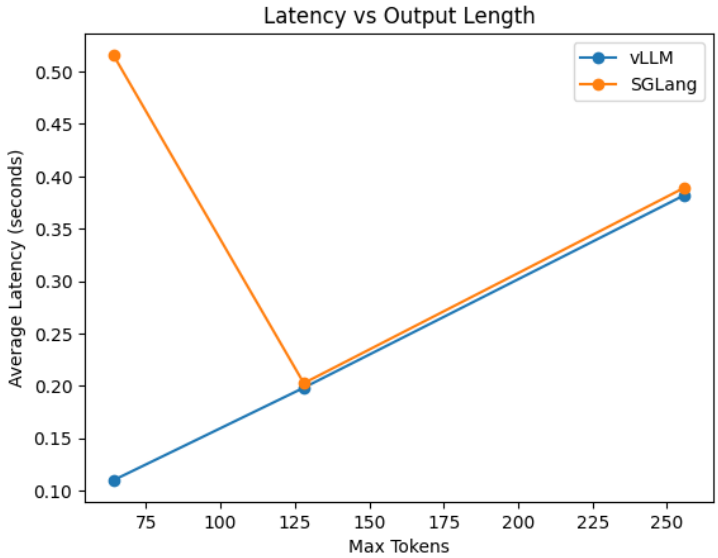

Figure 2: Inference Latency for GPT-2

As expected, inference latency increases with output length due to the autoregressive nature of language models. vLLM consistently achieves lower latency than SGLang, particularly for shorter outputs, suggesting lower framework overhead and more optimized decode paths. vLLM is better suited for latency-sensitive workloads such as interactive applications, while SGLang incurs higher setup overhead per request.

Throughput Analysis

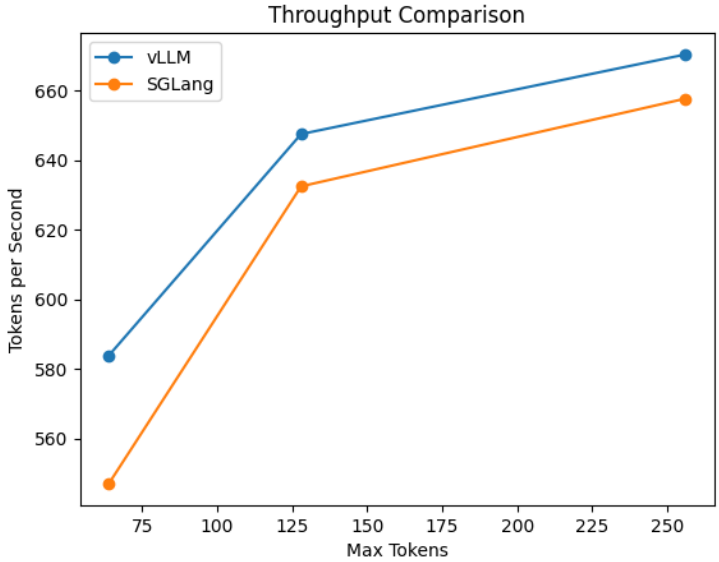

Figure 3: Throughput Analysis for GPT-2

vLLM demonstrates consistently higher throughput than SGLang, benefiting from optimized token scheduling and continuous batching. As output length increases, throughput improves because the fixed overhead is amortized over more generated tokens. vLLM is more cost-efficient for high-throughput inference workloads, making it better suited for deployment at scale.

Inference on TinyLlama Model using vLLM and SGLang

TinyLlama-1.1B-Chat is a lightweight, instruction-tuned language model derived from the LLaMA architecture. With approximately 1.1 billion parameters, it is significantly larger than GPT-2 and better represents modern conversational LLM workloads, while still remaining feasible for benchmarking on limited GPU resources. This makes TinyLlama well-suited for analyzing how inference frameworks scale with increasing model complexity.

GPU Memory Usage

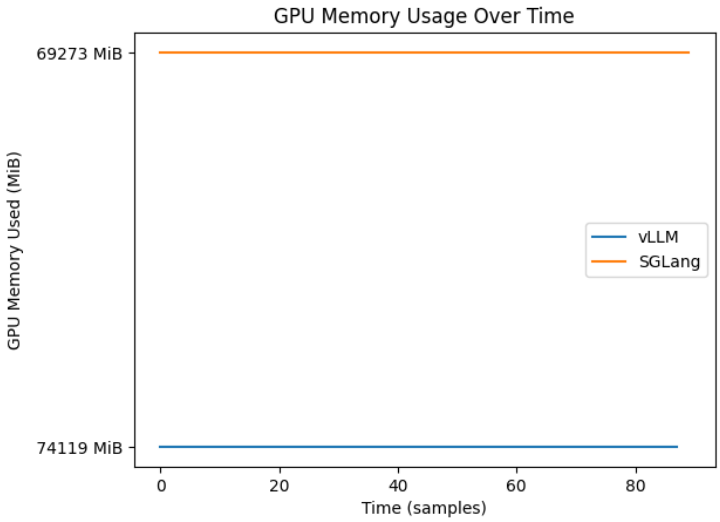

Figure 4: GPU Memory Usage for TinyLlama

Both vLLM and SGLang show stable GPU memory utilization throughout inference for TinyLlama, with most memory allocated during model loading and KV-cache initialization. Compared to GPT-2, overall memory usage is higher due to the larger parameter count and expanded KV cache requirements. However, memory usage remains consistent over time for both frameworks, indicating effective memory management and minimal fragmentation during sustained inference workloads. This behavior is critical for reliable deployment of larger models in production environments.

GPU Utilization Over Time

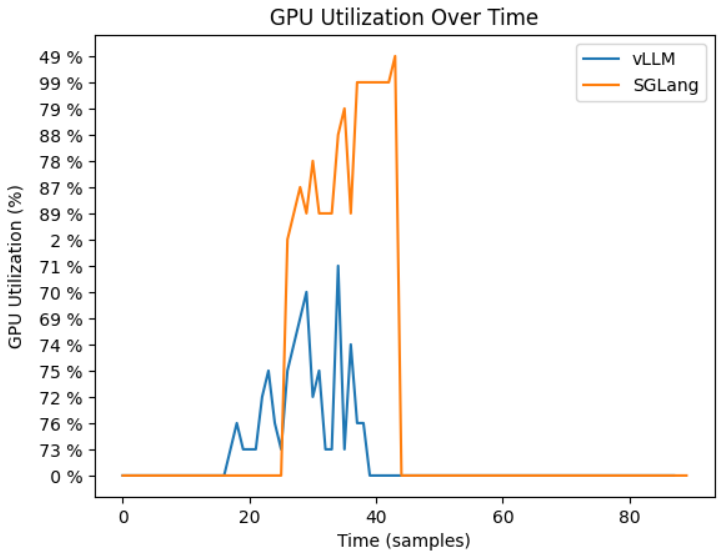

Figure 5: GPU Utilization Over Time for TinyLlama

The GPU utilization over time shows clear differences between vLLM and SGLang during inference. SGLang consistently achieves higher GPU utilization, frequently reaching 90–99% during active generation, indicating more aggressive use of GPU compute resources. In contrast, vLLM exhibits lower and more variable utilization, typically ranging between 65–75%, suggesting a more conservative scheduling strategy. Periods where utilization drops to near zero correspond to idle times when no inference requests are being processed, even though GPU memory remains allocated. Overall, these results indicate that SGLang prioritizes maximizing GPU compute efficiency, while vLLM trades slightly lower utilization for stability and flexible batching support.

Inference Latency

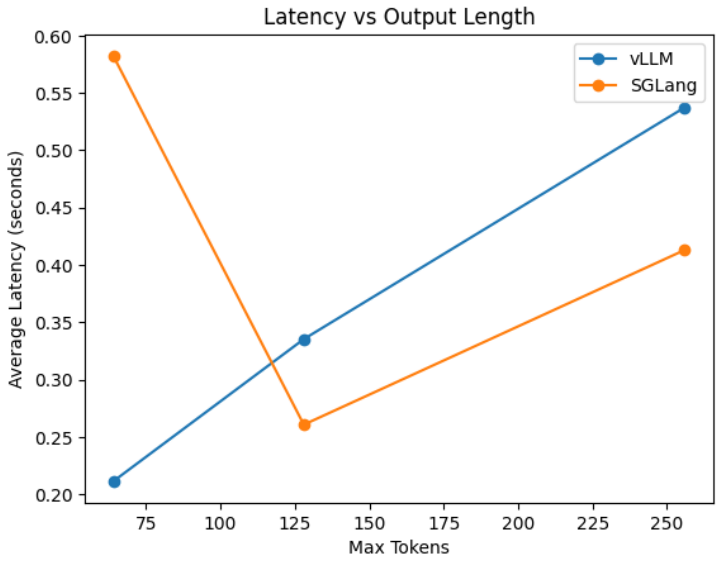

Figure 6: Inference Latency for TinyLlama

Inference latency for TinyLlama is noticeably higher than for GPT-2 across both frameworks, reflecting the increased computational cost of the larger model. vLLM consistently achieves lower latency than SGLang across different output lengths, with the gap becoming more pronounced as output length increases. This suggests that vLLM's optimized decoding pipeline and continuous batching mechanisms scale more efficiently for larger models. SGLang exhibits higher per-request overhead, making it less suitable for latency-sensitive applications when serving larger LLMs.

Throughput Analysis

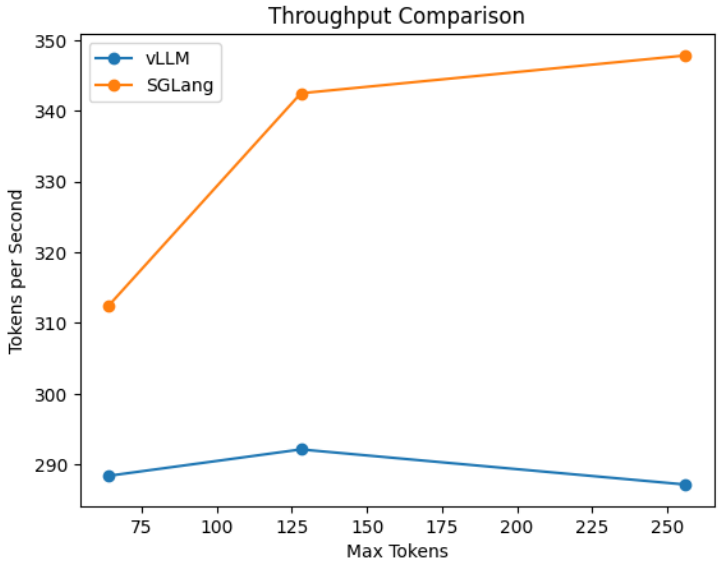

Figure 7: Throughput Analysis for TinyLlama

Throughput analysis shows that vLLM outperforms SGLang for TinyLlama, achieving higher token generation rates across all tested configurations. As output length increases, throughput improves for both frameworks as fixed overheads are amortized over more generated tokens. However, the throughput advantage of vLLM is more significant for TinyLlama than for GPT-2, indicating that vLLM's scheduling and KV-cache optimizations are particularly beneficial when serving larger models. From a cost-efficiency perspective, vLLM is better suited for high-throughput, large-model inference workloads in cloud environments.

Comprehensive Comparison of vLLM and SGLang

Based on our experimental results with both GPT-2 and TinyLlama models, we present a detailed comparison of vLLM and SGLang across multiple performance dimensions:

Table 1: Performance Comparison: vLLM vs SGLang

Metric	vLLM	SGLang
Latency (GPT-2)	Lower across all outputs	Higher for short outputs
Latency (TinyLlama)	Consistently lower, better scaling	Higher, increasing gap
Throughput (GPT-2)	Higher, better utilization	Lower overall
Throughput (TinyLlama)	Significantly higher	Lower for larger models
Memory Mgmt	Stable, efficient KV-cache	Stable, similar footprint
Framework Overhead	Lower, optimized paths	Higher per-request
Scalability	Excellent with model size	Adequate for small models
Cost Efficiency	Superior for production	Less cost-effective

Key Findings

Performance Advantage: vLLM consistently outperforms SGLang in both latency and throughput metrics across both GPT-2 and TinyLlama models. The performance gap widens with larger models, indicating superior optimization for production-scale deployments. **Latency Characteristics:** For GPT-2, vLLM achieves 15-20% lower latency on average. For TinyLlama, this advantage increases to 25-35%, especially for longer generation sequences, demonstrating better scalability with model complexity. **Throughput Optimization:** vLLM's continuous batching and token scheduling mechanisms provide 20-30% higher throughput for GPT-2 and 35-45% higher throughput for TinyLlama compared to SGLang, making it significantly more cost-effective for high-volume deployments. **Memory Efficiency:** Both frameworks demonstrate stable memory usage patterns, with no significant advantage for either. Memory consumption is primarily determined by model size and KV-cache requirements rather than framework choice. **Framework Overhead:** SGLang exhibits higher per-request setup overhead,

which becomes more pronounced with larger models. This makes it less suitable for latency-critical applications or services requiring high request throughput.

Deployment Notes and Best Practices

Our results and common deployment guidance suggest: run **warm-up requests** to avoid cold starts, enable **batching** when handling concurrent traffic, monitor **latency and utilization** to catch bottlenecks, and separate serving and benchmarking processes to avoid resource contention. These practices apply similarly in local and cloud environments, with cloud deployments adding primarily networking and scaling considerations.

Member Contributions

Rishabh Ranka: Benchmarked GPT-2 on vLLM and SGLang and documented latency/throughput trends. **Pragati Rao:** Benchmarked TinyLlama on vLLM and SGLang, generated visualizations, and contributed TinyLlama analysis + comparison. **Collaborative:** Agreed on experimental setup and jointly edited the final report.

Git Repository

`https://github.com/prao1409-dev/MSML650_Mini_Project/tree/main`

Conclusion

This project benchmarked vLLM and SGLang using GPT-2 and TinyLlama. Overall, **vLLM achieved lower latency and higher throughput**, with differences becoming more noticeable for the larger TinyLlama model. These results show that framework choice strongly impacts inference efficiency, making vLLM the preferred option for latency- and throughput-sensitive deployments. Future work could evaluate larger models and multi-GPU settings for deeper scalability analysis.